Distinguished Lecture Series III: Avi Wigderson, "Algebraic computation"

Terence Tao · 13 January 2008

https://terrytao.wordpress.com/2008/01/13/distinguished-lecture-series-iii-avi-wigderson-algebraic-computation/

tao2typ · converted to Typst 2026-10-14 (UTC)

Avi Wigderson's final talk in his Distinguished Lecture Series on "Computational complexity" was entitled "Arithmetic computation"; the complexity theory of *arithmetic* circuits rather than boolean circuits.

Arithmetic circuits manipulate inputs and outputs in a given field, such as the complex numbers (or more generally, a field of characteristic 0, to avoid having to deal with arithmetic "shortcuts" coming from identities such as $x^2 = x$ in $\mathbb{F}_2$). There are many models for such circuits, but one simple one is to create circuits by composing addition gates (which take two inputs x and y and return x+y as an output), multiplication gates (which take two inputs x, y and return xy as an output), and scalar multiplication gates (which take one input x and return cx as output, where c is a fixed constant for each such gate). Despite the name, circuits are not allowed to contain loops; gates can only use the outputs provided by earlier gates, although one is definitely allowed to reuse a given output for multiple future gates.

For simplicity let us view the scalar multiplication gates as being "free" and only count the addition and multiplication gates when counting complexity. Thus, for instance, to perform an inner product computation

$$(x_1, ..., x_n), (y_1, ..., y_n) \mapsto x_1 y_1 + ... + x_n y_n$$

requires 2n-1 gates (n to multiply the pairs x_i and y_i together, and then n-1 to add together the resulting pairs). It is not difficult to show that this 2n-1 is sharp. More generally, given any polynomial operation p with many inputs $x_1, ..., x_n$ and one output, we define the circuit complexity S(p) of p to be the least number of gates needed to build an arithmetic circuit that can generate p from $x_1, ..., x_n$; it is clear that p has to be polynomial in order to be generated by such circuits. We can also treat the complexity $S(p_1, ..., p_k)$ of multiple polynomials of some variables $x_1, ..., x_n$ by considering the least number of gates required to build a circuit that can generate all outputs from the given inputs. A good example here is matrix multiplication $A, B \mapsto AB$ of two $n \times n$ matrices A, B, in which there are $2n^2$ inputs (the coefficients of A and B) and n^2 outputs (the coefficients of AB. The obvious matrix multiplication algorithm requires a circuit of complexity $O(n^3)$, thus $S(AB) = O(n^3)$. But this is not best possible; the fast matrix multiplication methods of Strassen and later authors use clever recursive techniques to reduce the exponent, with the current record being $S(AB) = O(n^{2.38...})$. Conversely, it is clear that S(AB) should be at least n^2, since each of the inputs must be connected to at least one gate. But the true order of S(AB) is not known.

More generally, the problem of computing S(p) or $S(p_1, ..., p_k)$ for various interesting polynomials is an important one, to which there are still only very partial answers. Even the complexity of a single polynomial p(x) of a single variable x is not easy. If p has degree d, then clearly $S(p) = O(d)$, and counting arguments can show that this is sharp for "generic" p, but for many p one can do a lot better; for instance it is not hard to see that $S(x^d) = O(\log d)$ by repeatedly squaring x and then combining some of the iterated squares to form x^d. On the other hand, it is clear that we have a lower bound $S(x^d) \gg \log d$, since a circuit of m gates can only generate a polynomial of degree at most 2^m. (Here we see the usefulness of the degree concept in arithmetic circuit complexity; there does not seem to be an analogously useful concept in boolean circuit complexity, thus making the former subject easier than the latter.)

Avi noted, though, that it was still an open question to find an "natural" polynomial p of degree d for which S(p) grew faster than any power of $\log d$. (One could take a polynomial p whose coefficients are all algebraically independent, but this is "cheating".) In particular, it is not known if

$$S((x+1)...(x+d)) \gg \log^C d \tag{1}$$

for every C. But... if this claim failed, then one can show that factoring n-bit numbers can be done in polynomial time! (The reason is that if (1) failed, then one can compute d! mod N quickly for n-bit integers d and N by using a polynomial size circuit modulo N, which basically allows one to determine whether all prime factors of N are less than any given threshold d, from which one can quickly isolate each individual prime factor.) But this already shows how difficult we expect these problems to be in general.

Here are some other interesting examples of polynomials whose complexity we would like to understand:

1. The determinant polynomial $A \mapsto \det(A)$ on n^2 variables. It can be shown that $S(\det) = O(n^3)$ by Gaussian elimination (admittedly, this procedure uses divisions, but there is a way to rework it so that divisions are avoided – due to Strassen, apparently, though I don't have the reference), and by using fast matrix multiplication methods one can shave this down to $O(n^{2.38...})$ as before. Conversely, the trivial lower bound on complexity here is $\gg n^2$.
2. The permanent $A \mapsto \mathrm{per}(A)$ on n^2 variables; Avi referred to the permanent as the determinant's "bad cousin". Here, the best known upper bound for $S(\mathrm{per})$ is something like $2^{O(n)}$. The trivial lower bound is $\gg n^2$. Closing the exponential gap here is a major challenge, which Avi returned to later in the lecture.
3. The symmetric polynomial $\mathrm{Sym}^n_d(x_1, ..., x_n) := \sum_{1 \leq i_1 < ... < i_d \leq n} x_{i_1} ... x_{i_d}$ of degree d on n variables. By recursively describing the symmetric polynomials of degree up to d in terms of the same polynomials on one fewer variable, one can obtain an upper bound $S(\mathrm{Sym}^n_d) = O(nd)$. The trivial lower bound is $\gg n$.

It is not known whether any of the above upper bounds are sharp. But Baur and Strassen have shown some logarithmic improvements on some of the lower bounds. For instance:

1. $S(\mathrm{Sym}^n_d) \gg n \log d$;
2. $S(\det) \gg n^2 \log n$;
3. $S(x_1^d + ... + x_n^d) \gg n \log d$.
4. $S(x_1^d, ..., x_n^d) \gg n \log d$.

(Note that the bounds in (3) and (4) are tight.)

Let's first look at (4). This looks easy: after all; each individual polynomial x_i^d requires $\gg \log d$ gates to compute, and the polynomials are clearly so "independent" that there should be no advantage in sharing computation, leading to a lower bound $\gg n \log d$. But this naive argument is not valid. To see this, consider the problem of computing a product Ax of a constant $n \times n$ matrix A (with all entries algebraically independent) and a vector
x of n unknown entries. Because this computation involves n^2 independent constants, it must require $\gg n^2$ gates. Now if $x_1, ..., x_n$ are n vectors of n distinct unknowns each (thus having n^3 variables in all), the same reasoning as before would suggest that computing $Ax_1, ..., Ax_n$ simultaneously should require $\gg n^3$ gates. But the fast matrix multiplication algorithms imply that we can in fact rely on just $O(n^{2.38...})$ gates instead! So one's naive intuition on this subject can sometimes be wrong; there can be clever ways to save computation even when no obvious opportunities for such savings exist.

Due to the central role of polynomials in arithmetic complexity, it is not surprising that algebraic geometry plays an important role in the subject. For instance, the basic concept of the *degree* of a variety (or more precisely, an algebraic set) $V(p_1, ..., p_m) := \{p_1 = ... = p_m = 0\}$, which can be defined for instance (at least in the case where the underlying field is algebraically closed) as the maximum cardinality that an intersection of that variety with an affine subspace of the complementary dimension can have.

Thus for instance a line has degree 1, a conic section has degree 2, an elliptic curve has degree 3, and so forth. A basic result from algebraic geometry (which generalises Bezout's theorem) is that the degree $\deg(V \cap W)$ of the intersection of two algebraic sets is bounded by the product $\deg(V) \deg(W)$ of the individual degrees. Also, the affine projection of an algebraic set of degree at most d, is also an algebraic set of degree at most d. From these facts we see that the set of all possible input and output states of a circuit of complexity m form a variety of degree at most 2^m, since each addition gate determines a variety of degree 1 and each multiplication gate determines a variety of degree 2. On the other hand, one can easily check from the definition that the variety $\{x_1^d - 1 = ... = x_n^d - 1 = 0\}$ has degree d^n, giving a proof of (4) (note that the complexity of $x_1^d - 1, ..., x_n^d - 1$ and $x_1^d, ..., x_n^d$ clearly differ by at most O(n)).

The same argument does not directly work for (3), but there is a trick based on the upper bound

$$S(\frac{\partial p}{\partial x_1}, ..., \frac{\partial p}{\partial n}) \ll S(p) \ (*)$$

which allows one to deduce (3) from (4). (This is an example of how an upper bound on complexity can be used to deduce lower bounds from other lower bounds, which is a common technique in this business.) To prove (*), what one does is take a circuit that computes p and observe that by recursively working backwards from late outputs of this circuit to early outputs, one can compute all the partial derivatives $\partial p/\partial y$ of the final output p with respect to the value of any intermediate output y (where we temporarily allow ourselves the ability to vary these intermediate variables at will), while only increasing the number of gates by a multiplicative constant.

A similar argument gives (2), based on first computing the complexity of the inverse operation $A \mapsto A^{-1}$ (here, of course, one has to allow division), which is closely connected to the derivatives of the determinant (via the cofactor expansion). Avi did not discuss the proof of (1), but I presume that it also uses similar techniques.

Making further progress on complexity of arbitrary arithmetic circuits like this beyond the logarithmic improvements over the trivial bound seems to be very challenging, so Avi then turned to the problem of circuit complexity of *depth-bounded* circuits, for which more progress has been made. Here, we allow the addition and multiplication gates to take in an arbitrary number of inputs (and in the case of addition gates, we allow arbitrary scalar multiplications, thus we can take an arbitrary linear combination of inputs in a single gate), but we organise the gates in levels, with the inputs at depth j being used to produce outputs at depth j+1, and limit the total depth to be a small number such as 2 or 3. For instance, with depth 1, one can only create linear combinations or products of inputs. With depth 2, one can create any polynomial output, but it is not hard to see that the complexity is comparable to the number of non-zero terms in that polynomial (unless it is a product of linear polynomials, in which case it is comparable to the number of terms in the product). Depth 3 is the first really interesting case – unless the polynomial p factorises into sparse factors, the most efficient way to use depth 3 circuits is is the linear combination of products of linear polynomials (though occasionally one might want to swap the order of addition and multiplication). Let us use $S_3(p)$ to denote the number of gates needed to compute p with a depth 3 circuit.

One expects to have quite large lower bounds on $S_3(p)$ for various interesting values of p. For instance, the depth 3 complexity of the determinant is expected to be exponentially large, though this remains open. But in some cases there are clever tricks to compute polynomials quicker than one might first expect. For instance, Ben-Or proved the bound $S_3(\mathrm{Sym}_n^d) \ll n^2$, which is surprising given that Sym_n^d has exponentially many terms (if d is, say, equal to n/2) and also has no obvious factorisation. Actually, this is quite easy to show: with $O(n^2)$ gates in a depth 2 circuit one can already compute the quantities $P(t) := \prod_{j=1}^n (x_j + t)$ for $t = 0, ..., n$, and then one can recover the symmetric polynomials as the coefficients of P via polynomial interpolation. This turns out to be tight in the case when d is comparable to n, as shown by Shpilka and Wigderson.

Finally, some exponential-type lower bounds on complexity are known if one restricts one's circuits to not only have bounded depth, but also be *homogeneous* – which means all intermediate outputs need

to be homogeneous polynomials of the inputs. In particular, if one is computing a degree d output, then all multiplicative gates can have order at most d, thus ruling out the polynomial interpolation trick mentioned above (which requires degree n gates). Indeed, with this additional restriction on depth 3 circuits one can show that Sym_n^d now requires an exponential number of gates in n, if d is equal to (say) n/4. The idea here is to work not with the polynomial p *per se*, but rather with the linear space V_p of polynomials generated by its translations or (equivalently, by Taylor expansion) by its various derivatives. The dimension $\rho(p) := \dim(V_p)$ can easily be verified to obey the inequalities $\rho(p+q) \leq \rho(p) + \rho(q)$ and $\rho(pq) \leq \rho(p)\rho(q)$. In particular, if p is generated from a depth 3 circuit as the sum of m products of at most d linear polynomials, one has $\rho(p) \leq m2^d$. On the other hand, by inspecting how linearly independent the $d/2^{th}$ derivatives of Sym_n^d are, we have the bound $\rho(\mathrm{Sym}_n^d) \geq \binom{n-d/2}{d/2}$. Combining the two we obtain the exponential lower bound.

Finally, Avi turned to the permanent. Valiant observed that the permanent of an $n \times n$ matrix A can be re-expressed as the determinant of an $m \times m$ matrix B that depends linearly on A. Unfortunately, in his construction, m is exponentially large in n, and so the best circuit complexity upper bound on the permanent is similarly exponential. Using some algebraic geometry and some Hessian-type objects, Mignon-Ressayre observed that the minimal value of m one could take here had to be at least n^2. In contrast, Valiant proved that if one could obtain a lower bound of the form $m \gg n^C$ for all C, then one necessarily has (an arithmetic version of) $P \neq NP$! Thus we see that these apparently innocuous arithmetic complexity questions are intimately related to the rest of computational complexity theory.

[*Update*, Jan 16: slight change to the degree argument for lower bounding $S(x_1^d, \ldots, x_n^d)$.]

Comments (5)



Iftikhar Burhanuddin · 14 January 2008

I've made available the audio recordings and pictures of Avi Wigderson's DLS series of talks here: http://www.math.ucla.edu/~ntg/#seminar



Jason · 15 January 2008

Hi Terry,
Can you show in more detail why $S(Sym_n^d) = O(nd)$? I am still not clear from your hints that "By recursively describing the symmetric polynomials of degree up to d in terms of the same polynomials on one fewer variable, one can obtain an upper bound $S(Sym_n^d) = O(nd)$"
As I see it,
$Sym_n^d = Sym_{n-1}^d + x_n Sym_{n-1}^{d-1}$.
but can this recurrence result in an $O(nd)$ size circuit?
Moreover, since I'm computer science major, the notion "algebraic geometry" terrifies me. Can you provide any good textbooks about it, in particular suitbable for non-math major students?



Terence Tao · 15 January 2008

Dear Jason,
Using the recurrence described above, one can get from $\mathrm{Sym}_{n-1}^0, \ldots, \mathrm{Sym}_{n-1}^d$ to $\mathrm{Sym}_n^0, \ldots, \mathrm{Sym}_n^d$ using a circuit of complexity O(d). Iterating this n times gives the claim.
I learned algebraic geometry from a number of texts, from which I found Harris's text to be the most accessible. But perhaps others will have further suggestions.



Eric · 16 January 2008

“Algebraic complexity theory”, by Buergisser, Clausen, Shokrollahi is a good reference. It contains most of the algebra needed for the degree bound (by the way, the variety {x1^d=…=xn^d=0} has degree 1, since it’s a point)



Terence Tao · 16 January 2008

Thanks for the reference and the correction! It’s easy enough to fix, of course, by shifting all the x_j^d by 1, which I’ve now done.